

ANEXO SOBRE SQL

Como hemos visto, las instrucciones **DML** (Data Manipulation Language – Lenguaje de Manipulación de Datos) trabajan sobre los datos almacenados en nuestro SGBD, permitiendo consultarlos o modificarlos.

En general a las operaciones básicas de manipulación de datos que podemos realizar con SQL se les denomina **operaciones CRUD** (de **C**reate, **R**ead, **U**ppdate and **D**elete, o sea, Crear, Leer, Actualizar y Borrar, sería CLAB en español, pero no se usa). Lo verás utilizado de esta manera en muchos sitios, así que apréndete ese acrónimo.

Hay cuatro instrucciones para realizar estas tareas:

INSERT: Inserta filas en una tabla. Se corresponde con la “C” de CRUD.

SELECT: muestra información sobre los datos almacenados en la base de datos.

Dicha información puede pertenecer a una o varias tablas. Es la “R”.

UPDATE: Actualiza información de una tabla. Es, obviamente, la “U”.

DELETE: Borra filas de una tabla. Se corresponde con la “D”.

Consulta de datos

Ahora nos vamos a centrar en la “R” de CRUD, es decir, en **cómo recuperar la información que nos interesa de dentro de una base de datos**, usando para ello el lenguaje de consulta o SQL. Ya nos preocuparemos luego de cómo llegamos a introducir los datos primeramente.

Para realizar consultas sobre las tablas de las bases de datos disponemos de la instrucción **SELECT**. Con ella podemos consultar una o varias tablas. Es sin duda **el comando más versátil del lenguaje SQL**.

Existen muchas cláusulas asociadas a la sentencia SELECT (GROUP BY, ORDER, HAVING, UNION). También es una de las instrucciones en la que con más frecuencia los motores de bases de datos incorporan cláusulas adicionales al estándar, que es el que veremos aquí.

Vamos a empezar viendo las consultas simples, basadas en una sola tabla. Veremos cómo obtener filas y columnas de una tabla en el orden en que nos haga falta. El resultado de una consulta SELECT nos devuelve **una tabla lógica**. Es decir, los resultados son una relación de datos, que tiene filas/registros, con una serie de campos/columnas. Igual que cualquier tabla de la base de datos. Sin embargo esta tabla está en memoria mientras la utilizamos, y luego se descarta. Cada vez que ejecutamos la consulta se vuelve a calcular el resultado.

La sintaxis básica de una consulta SELECT es la siguiente (los valores opcionales van entre corchetes):

```
SELECT [ ALL / DISTINC ] [ * ] / [ListaColumnas_Expresiones] AS
[Expresion]
FROM Nombre_Tabla_Vista
WHERE Condiciones
ORDER BY ListaColumnas [ ASC / DESC ]
```

A continuación analizaremos cada una de las partes de la consulta para entenderla mejor.

SELECT

Permite seleccionar las columnas que se van a mostrar y en el orden en que lo van a hacer. Simplemente es la instrucción que la base de datos interpreta como que vamos a solicitar información.

ALL / DISTINCT

ALL es el valor predeterminado, especifica que el conjunto de resultados puede incluir filas duplicadas. Por regla general nunca se utiliza.

DISTINCT especifica que el conjunto de resultados sólo puede incluir filas únicas. Es decir, si al realizar una consulta hay registros exactamente iguales que aparecen más de una vez, éstos se eliminan. Muy útil en muchas ocasiones.

Nombres de campos

Se debe especificar una lista de nombres de campos de la tabla que nos interesan y que por tanto queremos devolver. Normalmente habrá más de uno, en cuyo caso separamos cada nombre de los demás mediante comas.

Se puede anteponer el nombre de la tabla al nombre de las columnas, utilizando el formato Tabla.Columna. Además de nombres de columnas, en esta lista se pueden poner constantes, expresiones aritméticas, y funciones, para obtener campos calculados de manera dinámica.

Si queremos que nos devuelva todos los campos de la tabla utilizamos el comodín “*” (asterisco).

Los nombres indicados deben coincidir exactamente con los nombre de los campos de la tabla, pero si queremos que en nuestra tabla lógica de resultados tengan un nombre diferente podemos utilizar:

AS

Permite renombrar columnas si lo utilizamos en la cláusula SELECT, o renombrar tablas si lo utilizamos en la cláusula FROM. Es opcional. Con ello podremos crear diversos alias de columnas y tablas. Enseguida veremos un ejemplo.

FROM

Esta cláusula permite indicar las tablas o vistas de las cuales vamos a obtener la información. De momento veremos ejemplos para obtener información de una sola tabla. Como se ha indicado anteriormente, también se pueden renombrar las tablas usando la instrucción “AS”.

WHERE

Especifica la **condición de filtro** de las filas devueltas. Se utiliza cuando no se desea que se devuelvan todas las filas de una tabla, sino sólo las que cumplen ciertas condiciones. Lo habitual es utilizar esta cláusula en la mayoría de las consultas.

Condiciones

Son **expresiones lógicas** a comprobar para la condición de filtro, que tras su resolución devuelven para cada fila TRUE o FALSE, en función de que se cumplan o no. Se puede utilizar cualquier expresión lógica y en ella utilizar **diversos operadores** como:

- > (Mayor)
- >= (Mayor o igual)
- < (Menor)
- <= (Menor o igual)
- = (Igual)
- <> o != (Distinto)

IS [NOT] NULL (para comprobar si el valor de una columna es o no es nula, es decir, si contiene o no contiene algún valor).

Se dice que una columna de una fila es NULL si está completamente vacía. Hay que tener en cuenta que si se ha introducido cualquier dato, incluso en un campo alfanumérico si se introduce una cadena en blanco o un cero en un campo numérico, deja de ser NULL.

LIKE: para la comparación de un modelo. Para ello utiliza los caracteres comodín especiales: “%” y “_”. Con el primero indicamos que en su lugar puede ir cualquier cadena de caracteres, y con el segundo que puede ir cualquier carácter individual (un solo carácter). Con la combinación de estos caracteres podremos obtener múltiples patrones de búsqueda.

Por ejemplo:

El nombre empieza por A: `Nombre LIKE 'A%'`

El nombre acaba por A: `Nombre LIKE '%A'`

El nombre contiene la letra A: `Nombre LIKE '%A%'`

El nombre empieza por A y después contiene un solo carácter cualquiera:

`Nombre LIKE 'A_'`

El nombre empieza una A, después cualquier carácter, luego una E y al final cualquier cadena de caracteres: `Nombre LIKE 'A_E%'`

BETWEEN: para un intervalo de valores.

Por ejemplo:

Clientes entre el 30 y el 100: `CodCliente BETWEEN 30 AND 100`

Clientes nacidos entre 1970 y 1979: `FechaNac BETWEEN '19700101' AND '19791231'`

IN(): para especificar una relación de valores concretos. Por ejemplo: Ventas de los Clientes 10, 15, 30 y 75: `CodCliente IN(10, 15, 30, 75)`

Por supuesto es posible combinar varias condiciones simples de los operadores anteriores utilizando los operadores lógicos **OR**, **AND** y **NOT**, así como el uso de paréntesis para controlar la prioridad de los operadores (como en matemáticas). Por ejemplo: ... `(Cliente = 100 AND Provincia = 30) OR Ventas > 1000` ... que sería para los clientes de las provincias 100 y 30 o cualquier cliente cuyas ventas superen 1000.

ORDER BY

Define el orden de las filas del conjunto de resultados. Se especifica el campo o campos (separados por comas) por los cuales queremos ordenar los resultados.

ASC / DESC

ASC es el valor predeterminado, especifica que la columna indicada en la cláusula ORDER BY se ordenará de forma ascendente, o sea, de menor a mayor. Si por el contrario se especifica DESC se ordenará de forma descendente (de mayor a menor). Por ejemplo, para ordenar los resultados de forma ascendente por ciudad, y los que sean de la misma ciudad de forma descendente por nombre, utilizaríamos esta cláusula de ordenación:

```
... ORDER BY Ciudad, Nombre DESC ...
```

Como a la columna Ciudad no le hemos puesto ASC o DESC se usará para la misma el valor predeterminado (que es ASC)

OJO: Aunque al principio si aún no se está habituado, pueda dar la impresión de que se ordena por ambas columnas en orden descendente. Si es eso lo que queremos deberemos escribir ... ORDER BY Ciudad DESC, Nombre DESC ...

Funciones de agregación

Las funciones de agregación en SQL nos permiten efectuar operaciones sobre un conjunto de resultados, pero devolviendo un único valor agregado para todos ellos. Es decir, nos permiten obtener medias, máximos, etc... sobre un conjunto de valores.

Las funciones de agregación básicas que soportan todos los gestores de datos son las siguientes:

COUNT: devuelve el número total de filas seleccionadas por la consulta.

MIN: devuelve el valor mínimo del campo que especifiquemos.

MAX: devuelve el valor máximo del campo que especifiquemos.

SUM: suma los valores del campo que especifiquemos. Sólo se puede utilizar en columnas numéricas.

AVG: devuelve el valor promedio del campo que especifiquemos. Sólo se puede utilizar en columnas numéricas.

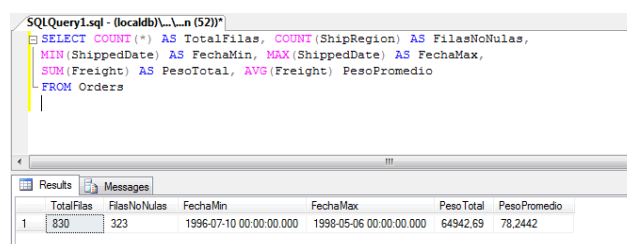
Las funciones anteriores son las básicas en SQL, pero cada sistema gestor de bases de datos relacionales ofrece su propio conjunto, más amplio, con otras funciones de agregación particulares. Puedes consultar las que ofrecen [SQL Server](#), [Oracle](#) y [MySQL](#).

Todas estas funciones se aplican a una sola columna, que especificaremos entre paréntesis, excepto la función COUNT, que se puede aplicar a una columna o indicar un "*". La diferencia entre poner el nombre de una columna o un "*", es que en el primer caso no cuenta los valores nulos para dicha columna, y en el segundo sí.

Nota: Si esta serie de artículos te está pareciendo interesante, entonces ni te imaginas lo que puedes aprender [con este curso de fundamentos de SQL](#).

Así, por ejemplo, si queremos obtener algunos datos agregados de la tabla de pedidos de la [base de datos de ejemplo Northwind](#), podemos escribir una consulta simple como la siguiente:

```
SELECT COUNT(*) AS TotalFilas, COUNT(ShipRegion) AS FilasNoNulas,  
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,  
SUM(Freight) AS PesoTotal, AVG(Freight) AS PesoPromedio  
FROM Orders
```



The screenshot shows a SQL query window with the following text:

```
SQLQuery1.sql - (localdb)\...\n (52) |  
SELECT COUNT(*) AS TotalFilas, COUNT(ShipRegion) AS FilasNoNulas,  
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,  
SUM(Freight) AS PesoTotal, AVG(Freight) AS PesoPromedio  
FROM Orders
```

Below the query window, the 'Results' tab is active, displaying the following data:

	TotalFilas	FilasNoNulas	FechaMin	FechaMax	PesoTotal	PesoPromedio
1	830	323	1996-07-10 00:00:00.000	1998-05-06 00:00:00.000	64942.69	78.2442

De esta manera sabremos que existen en total 830 pedidos en la base de datos, 323 registros que tienen asignada una zona de entrega, la fecha del pedido más antiguo (el 10 de julio de 1996), la fecha del pedido más reciente (el 6 de mayo de 1998 ¡los datos de ejemplo son muy antiguos!), el total de peso enviado entre todos los pedidos (64.942,69 Kg o sea, más de 64 toneladas) y el peso promedio del los envíos (78,2442Kg). No está mal para una consulta tan simple.

Como podemos observar del resultado de la consulta anterior, **las funciones de agregación devuelven una sola fila**, salvo que vayan unidas a la cláusula GROUP BY, que veremos a continuación.

Agrupando resultados

La cláusula **GROUP BY** unida a un **SELECT** permite **agrupar filas según las columnas que se indiquen como parámetros**, y se suele utilizar en conjunto con las funciones de agrupación, para **obtener datos resumidos y agrupados** por las columnas que se necesiten.

Hemos visto en el ejemplo anterior que obteníamos sólo una fila con los datos indicados **correspondientes a toda la tabla**. Ahora vamos a ver con otro ejemplo cómo obtener datos correspondientes a diversos grupos de filas, concretamente agrupados por cada empleado:

```
SELECT EmployeeID, COUNT(*) AS TotalPedidos, COUNT(ShipRegion) AS
FilasNoNulas,
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,
SUM(Freight) PesoTotal, AVG(Freight) PesoPromedio
FROM Orders
GROUP BY EmployeeID
```

En este caso obtenemos los mismos datos pero agrupándolos por empleado, de modo que para cada empleado de la base de datos sabemos cuántos pedidos ha realizado, cuándo fue el primero y el último, etc...:

SQLQuery2.sql - (localdb)\...n (53))* SQLQuery1.sql - (localdb)\...n (52))*

```
SELECT EmployeeID, COUNT(*) AS TodasFilas, COUNT(ShipRegion) AS FilasNoNulas,
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,
SUM(Freight) TotalPeso, AVG(Freight) Promedio
FROM Orders
GROUP BY EmployeeID
```

	EmployeeID	TodasFilas	FilasNoNulas	FechaMin	FechaMax	TotalPeso	Promedio
1	9	43	14	1996-07-15 00:00:00.000	1998-05-04 00:00:00.000	3326,26	77,3548
2	3	127	56	1996-07-15 00:00:00.000	1998-05-06 00:00:00.000	10884,74	85,7066
3	6	67	32	1996-07-10 00:00:00.000	1998-04-24 00:00:00.000	3780,47	56,4249
4	7	72	22	1996-08-28 00:00:00.000	1998-05-05 00:00:00.000	6665,44	92,5755
5	1	123	50	1996-07-23 00:00:00.000	1998-05-06 00:00:00.000	8836,64	71,8426
6	4	156	62	1996-07-11 00:00:00.000	1998-05-01 00:00:00.000	11346,14	72,7316
7	5	42	14	1996-07-16 00:00:00.000	1998-04-29 00:00:00.000	3918,71	93,3026
8	2	96	31	1996-08-12 00:00:00.000	1998-05-04 00:00:00.000	8696,41	90,5876
9	8	104	42	1996-07-25 00:00:00.000	1998-05-05 00:00:00.000	7487,88	71,9988

De hecho nos resultaría muy fácil cruzarla con la tabla de empleados, usando lo aprendido sobre consultas multi-tabla, y que se devolvieran los mismos resultados con el nombre y los apellidos de cada empleado:

SQLQuery2.sql - (localdb)\...n (53))*

```

SELECT Employees.FirstName + ' ' + Employees.LastName AS Empleado, COUNT(*) AS TotalPedidos,
COUNT(ShipRegion) AS FilasNoNulas,
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,
SUM(Freight) AS PesoTotal, AVG(Freight) AS PesoPromedio
FROM Orders INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
GROUP BY Employees.FirstName + ' ' + Employees.LastName

```

Results Messages

	Empleado	TotalPedidos	FilasNoNulas	FechaMin	FechaMax	PesoTotal	PesoPromedio
1	Andrew Fuller	96	31	1996-08-12 00:00:00.000	1998-05-04 00:00:00.000	8696,41	90,5876
2	Anne Dodsworth	43	14	1996-07-15 00:00:00.000	1998-05-04 00:00:00.000	3326,26	77,3548
3	Janet Leverling	127	56	1996-07-15 00:00:00.000	1998-05-06 00:00:00.000	10884,74	85,7066
4	Laura Callahan	104	42	1996-07-25 00:00:00.000	1998-05-05 00:00:00.000	7487,88	71,9988
5	Margaret Peacock	156	62	1996-07-11 00:00:00.000	1998-05-01 00:00:00.000	11346,14	72,7316
6	Michael Suyama	67	32	1996-07-10 00:00:00.000	1998-04-24 00:00:00.000	3780,47	56,4249
7	Nancy Davolio	123	50	1996-07-23 00:00:00.000	1998-05-06 00:00:00.000	8836,64	71,8426
8	Robert King	72	22	1996-08-28 00:00:00.000	1998-05-05 00:00:00.000	6665,44	92,5755
9	Steven Buchanan	42	14	1996-07-16 00:00:00.000	1998-04-29 00:00:00.000	3918,71	93,3026

En este caso fíjate en cómo hemos usado la expresión `Employees.FirstName + ' ' + Employees.LastName` como parámetro en `GROUP BY` para que nos agrupe por un campo compuesto (en SQL Server no podemos usar alias de campos para las agrupaciones). De esta forma tenemos casi un informe preparado con una simple consulta de agrupación.

Importante: Es muy importante tener en cuenta que cuando utilizamos la cláusula `GROUP BY`, los únicos campos que podemos incluir en el `SELECT` sin que estén dentro de una función de agregación, son los que vayan especificados en el `GROUP BY`.

La cláusula `GROUP BY` se puede utilizar con más de un campo al mismo tiempo. Si indicamos más de un campo como parámetro nos devolverá la información agrupada por los registros que tengan el mismo valor en los campos indicados.

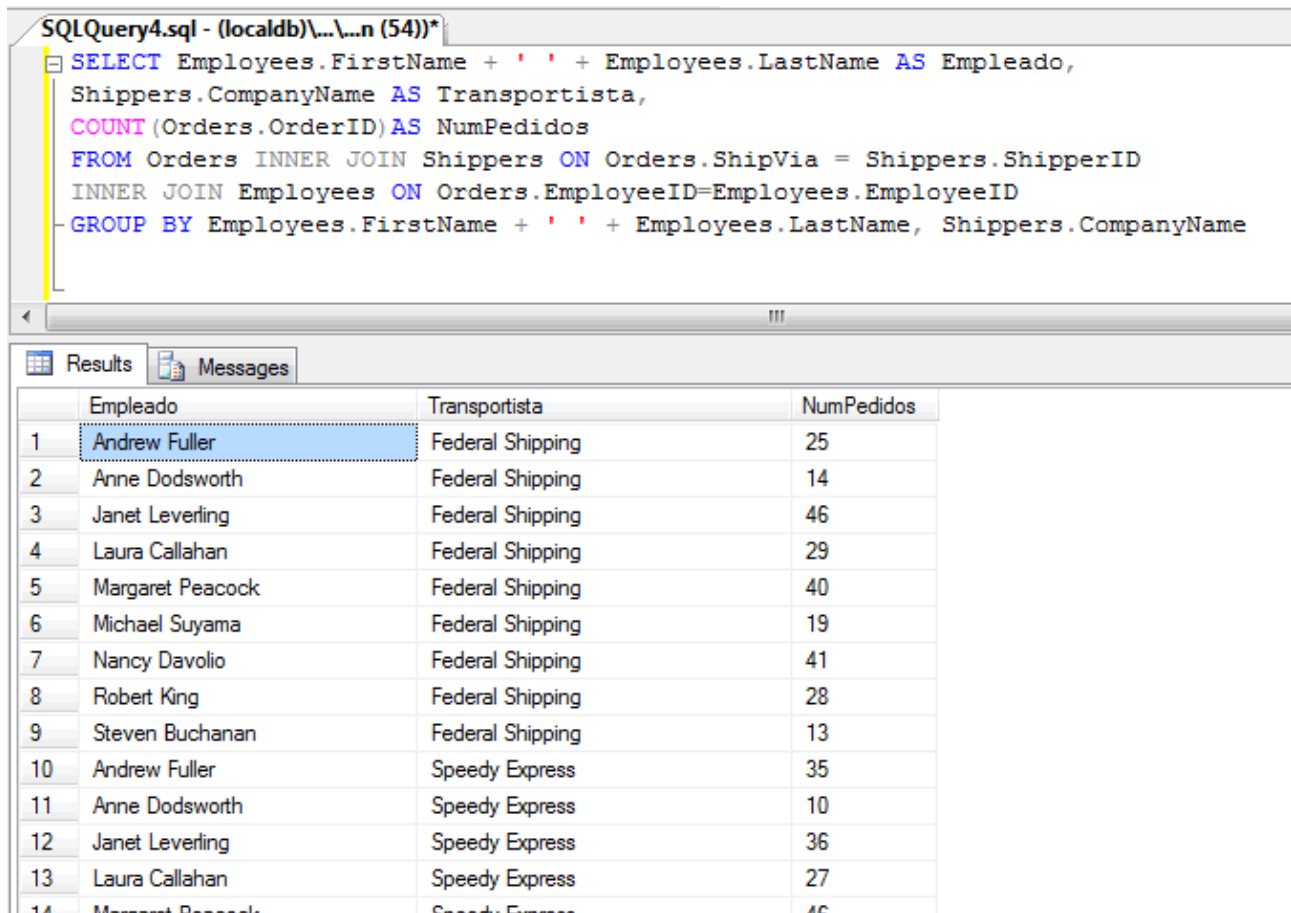
Por ejemplo, si queremos conocer la cantidad de pedidos que cada empleado ha enviado a través de cada transportista, podemos escribir una consulta como la siguiente:

```

SELECT Employees.FirstName + ' ' + Employees.LastName AS Empleado,
Shippers.CompanyName AS Transportista,
COUNT(Orders.OrderID) AS NumPedidos
FROM Orders INNER JOIN Shippers ON Orders.ShipVia =
Shippers.ShipperID
INNER JOIN Employees ON Orders.EmployeeID=Employees.EmployeeID
GROUP BY Employees.FirstName + ' ' + Employees.LastName,
Shippers.CompanyName

```

Con el siguiente resultado:



The screenshot shows a SQL query window titled 'SQLQuery4.sql - (localdb)\... (54)*'. The query is as follows:

```
SELECT Employees.FirstName + ' ' + Employees.LastName AS Empleado,  
Shippers.CompanyName AS Transportista,  
COUNT(Orders.OrderID) AS NumPedidos  
FROM Orders INNER JOIN Shippers ON Orders.ShipVia = Shippers.ShipperID  
INNER JOIN Employees ON Orders.EmployeeID=Employees.EmployeeID  
GROUP BY Employees.FirstName + ' ' + Employees.LastName, Shippers.CompanyName
```

Below the query window, the 'Results' tab is active, displaying a table with 4 columns: 'Empleado', 'Transportista', and 'NumPedidos'. The table contains 14 rows of data, showing the number of orders for each employee and shipper combination.

	Empleado	Transportista	NumPedidos
1	Andrew Fuller	Federal Shipping	25
2	Anne Dodsworth	Federal Shipping	14
3	Janet Leverling	Federal Shipping	46
4	Laura Callahan	Federal Shipping	29
5	Margaret Peacock	Federal Shipping	40
6	Michael Suyama	Federal Shipping	19
7	Nancy Davolio	Federal Shipping	41
8	Robert King	Federal Shipping	28
9	Steven Buchanan	Federal Shipping	13
10	Andrew Fuller	Speedy Express	35
11	Anne Dodsworth	Speedy Express	10
12	Janet Leverling	Speedy Express	36
13	Laura Callahan	Speedy Express	27
14	Margaret Peacock	Speedy Express	46

Así, sabremos que Andrew Fuller envió 25 pedidos con Federal Shipping, y 35 con Federal Express.

El utilizar la cláusula **GROUP BY** no garantiza que los datos se devuelvan **ordenados**. Suele ser una práctica recomendable **incluir una cláusula ORDER BY** por las mismas columnas que utilicemos en GROUP BY, especificando el orden que nos interesa. Por ejemplo, en el caso anterior

Existe una cláusula especial, parecida a la WHERE que ya conocemos que nos permite especificar las condiciones de filtro para los diferentes grupos de filas que devuelven estas consultas agregadas. Esta cláusula es **HAVING**.

HAVING es muy similar a la cláusula WHERE, pero en vez de afectar a las filas de la tabla, afecta a los grupos obtenidos.

Por ejemplo, si queremos repetir la consulta de pedidos por empleado de hace un rato, pero obteniendo solamente aquellos que hayan enviado más de 5.000 Kg de producto, y ordenados por el nombre del empleado, la consulta sería muy sencilla usando HAVING y ORDER BY:

```
SELECT Employees.FirstName + ' ' + Employees.LastName AS Empleado,  
COUNT(*) AS TotalPedidos,  
COUNT(ShipRegion) AS FilasNoNulas,  
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,
```

```

SUM(Freight) PesoTotal, AVG(Freight) PesoPromedio
FROM Orders INNER JOIN Employees ON Orders.EmployeeID =
Employees.EmployeeID
GROUP BY Employees.FirstName + ' ' + Employees.LastName
HAVING SUM(Freight) > 5000
ORDER BY Employees.FirstName + ' ' + Employees.LastName ASC

```

Ahora obtenemos los resultados agrupados por empleado también, pero solo aquellos que cumplan la condición indicada (o condiciones indicadas, pues se pueden combinar). Antes nos salían 9 empleados, y ahora solo 6 pues hay 3 cuyos envíos totales son muy pequeños:

SQLQuery2.sql - (localdb)\...n (53))*

```

SELECT Employees.FirstName + ' ' + Employees.LastName AS Empleado, COUNT(*) AS TotalPedidos,
COUNT(ShipRegion) AS FilasNoNulas,
MIN(ShippedDate) AS FechaMin, MAX(ShippedDate) AS FechaMax,
SUM(Freight) PesoTotal, AVG(Freight) PesoPromedio
FROM Orders INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
GROUP BY Employees.FirstName + ' ' + Employees.LastName
HAVING SUM(Freight) > 5000
ORDER BY Employees.FirstName + ' ' + Employees.LastName ASC

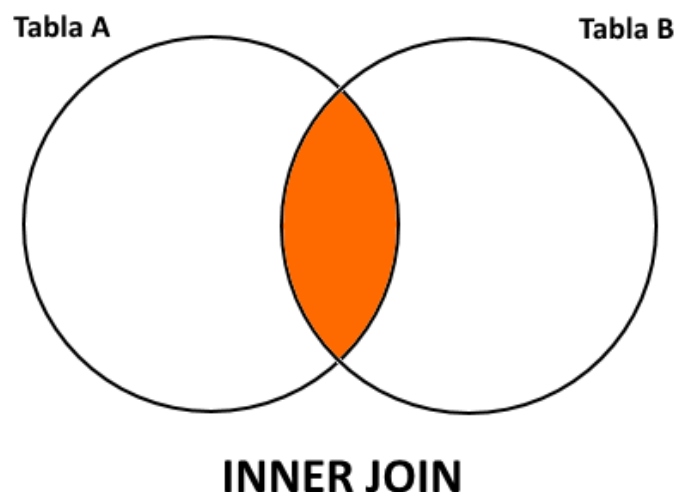
```

	Empleado	TotalPedidos	FilasNoNulas	FechaMin	FechaMax	PesoTotal	PesoPromedio
1	Andrew Fuller	96	31	1996-08-12 00:00:00.000	1998-05-04 00:00:00.000	8696,41	90,5876
2	Janet Leverling	127	56	1996-07-15 00:00:00.000	1998-05-06 00:00:00.000	10884,74	85,7066
3	Laura Callahan	104	42	1996-07-25 00:00:00.000	1998-05-05 00:00:00.000	7487,88	71,9988
4	Margaret Peacock	156	62	1996-07-11 00:00:00.000	1998-05-01 00:00:00.000	11346,14	72,7316
5	Nancy Davolio	123	50	1996-07-23 00:00:00.000	1998-05-06 00:00:00.000	8836,64	71,8426
6	Robert King	72	22	1996-08-28 00:00:00.000	1998-05-05 00:00:00.000	6665,44	92,5755

Combinaciones internas - INNER JOIN

Las combinaciones internas se realizan mediante la instrucción **INNER JOIN**. Devuelven únicamente aquellos registros/filas **que tienen valores idénticos en los dos campos** que se comparan para unir ambas tablas. Es decir aquellas que tienen elementos en las dos tablas, identificados éstos por el campo de relación.

La mejor forma de verlo es con un **diagrama de Venn** que ilustre en qué parte de la relación deben existir registros:



En este caso se devuelven los registros que tienen nexo de unión en ambas tablas. Por ejemplo, en la relación entre las tablas de clientes y pedidos en [Northwind](#), se devolverán los registros de todos los clientes que tengan al menos un pedido, relacionándolos por el ID de cliente.

Esto **puede ocasionar la desaparición del resultado de filas** de alguna de las dos tablas, por tener valores nulos, o por tener un valor que no exista en la otra tabla entre los campos/columnas que se están comparando.

Su sintaxis es:

```
FROM Tabla1 [INNER] JOIN Tabla2 ON Condiciones_Vinculos_Tablas
```

Así, para seleccionar los registros comunes entre la Tabla1 y la Tabla2 que tengan correspondencia entre ambas tablas por el campo Col1, escribiríamos:

```
SELECT T1.Col1, T1.Col2, T1.Col3, T2.Col7  
FROM Tabla1 T1 INNER JOIN Tabla2 T2 ON T1.Col1 = T2.Col1
```

Por ejemplo, para obtener en Northwind los clientes que tengan algún pedido, bastaría con escribir:

```
SELECT OrderID, C.CustomerID, CompanyName, OrderDate  
FROM Customers C INNER JOIN Orders O ON C.CustomerID =  
O.CustomerID
```

El cual nos nos devolverá 830 registros. Hay dos pedidos en la tabla de Orders sin cliente asociado, como puedes comprobar, y éstos no se devuelven por no existir la relación entre ambas tablas.

En realidad esto ya lo conocíamos puesto que en las combinaciones internas, el uso de la palabra **INNER** es opcional (por eso lo hemos puesto entre corchetes). Si simplemente indicamos la palabra JOIN y la combinación de columnas (como ya hemos visto en el artículo anterior) el sistema sobreentiende que estamos haciendo una combinación interna. Lo hemos incluido por ampliar la explicación y por completitud.

Combinaciones externas (OUTER JOIN)

Las **combinaciones externas** se realizan mediante la instrucción **OUTER JOIN**. Como enseguida veremos, devuelven todos los valores de la tabla que hemos puesto a la derecha, los de la tabla que hemos puesto a la izquierda o los de ambas tablas según el caso, devolviendo además valores nulos en las columnas de las tablas que no tengan el valor existente en la otra tabla.

Es decir, que **nos permite seleccionar algunas filas de una tabla aunque éstas no tengan correspondencia con las filas de la otra tabla** con la que se combina. Ahora lo veremos mejor en cada caso concreto, ilustrándolo con un diagrama para una mejor comprensión.

La sintaxis general de las combinaciones externas es:

```
FROM Tabla1 [LEFT/RIGHT/FULL] [OUTER] JOIN Tabla2 ON  
Condiciones_Vinculos_Tablas
```

Como vemos existen tres variantes de las combinaciones externas.

En todas estas combinaciones externas **el uso de la palabra OUTER es opcional**. Si utilizamos LEFT, RIGHT o FULL y la combinación de columnas, el sistema sobreentiende que estamos haciendo una combinación externa.

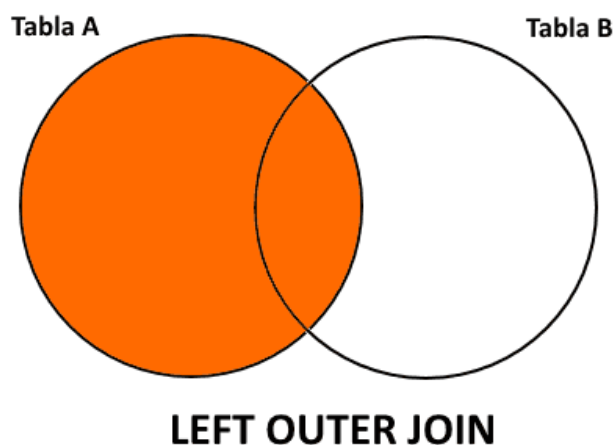
Variante LEFT JOIN

Se obtienen todas las filas de la tabla colocada a la izquierda, aunque no tengan correspondencia en la tabla de la derecha.

Así, para seleccionar todas las filas de la Tabla1, aunque no tengan correspondencia con las filas de la Tabla2, suponiendo que se combinan por la columna Col1 de ambas tablas escribiríamos:

```
SELECT T1.Col1, T1.Col2, T1.Col3, T2.Col7  
FROM Tabla1 T1 LEFT [OUTER] JOIN Tabla2 T2 ON T1.Col1 = T2.Col1
```

Esto se ilustra gráficamente de la siguiente manera:



De este modo, volviendo a Northwind, si escribimos la siguiente consulta:

```
SELECT OrderID, C.CustomerID, CompanyName, OrderDate  
FROM Customers C LEFT JOIN Orders O ON C.CustomerID = O.CustomerID
```

Obtendremos 832 registros ya que se incluyen **todos los clientes y sus pedidos, incluso aunque no tengan pedido alguno**. Los que no tienen pedidos carecen de la relación apropiada entre las dos tablas a partir del campo CustomerID. Sin embargo se añaden al resultado final dejando la parte correspondiente a los datos de la tabla de pedidos con valores nulos, como se puede ver en esta captura de SQL Server:

</

Variante RIGHT JOIN

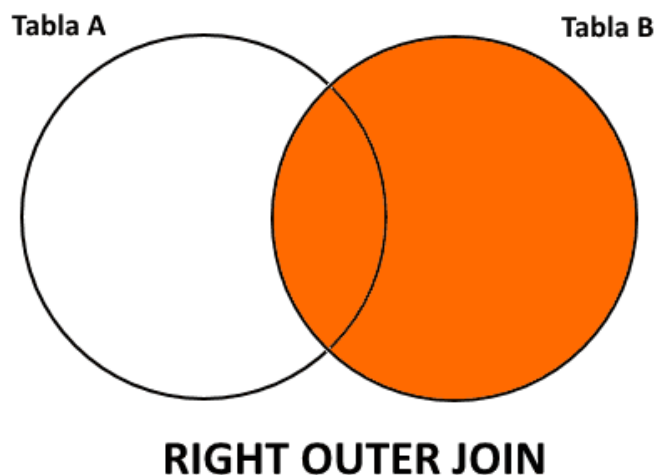
Análogamente, usando RIGHT JOIN se obtienen todas las filas de la tabla de la derecha, aunque no tengan correspondencia en la tabla de la izquierda. Así, para seleccionar todas las filas de la Tabla2, aunque no tengan correspondencia con las filas de la Tabla1 podemos utilizar la cláusula RIGHT:

```

SELECT T1.Col1, T1.Col2, T1.Col3, T2.Col7
FROM Tabla1 T1 RIGHT [OUTER] JOIN Tabla2 T2 ON T1.Col1 = T2.Col1

```

El diagrama en este caso es complementario al anterior:



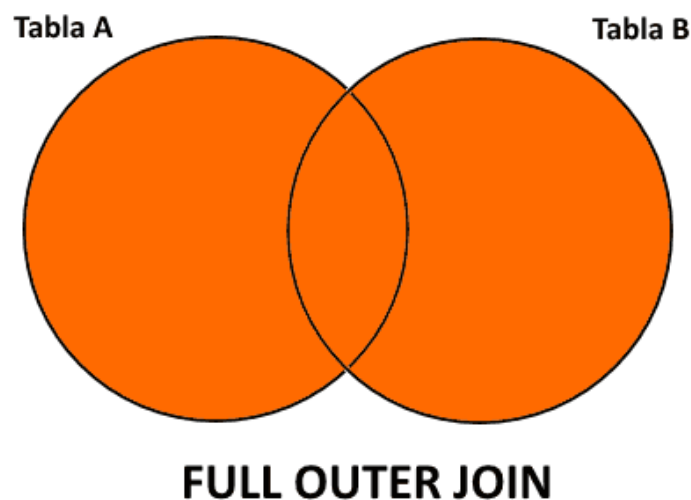
Si en nuestra base de datos de ejemplo queremos obtener **todos los pedidos aunque no tengan cliente asociado**, junto a los datos de dichos clientes, escribiríamos:

```
SELECT OrderID, C.CustomerID, CompanyName, OrderDate
FROM Customers C RIGHT JOIN Orders O ON C.CustomerID =
O.CustomerID
```

En este caso se devuelven 830 registros que son todos los pedidos. Si hubiese algún pedido con el CustomerID vacío (nulo) se devolvería también en esta consulta (es decir, órdenes sin clientes), aunque en la base de datos de ejemplo no se da el caso.

Variante **FULL JOIN**

Se obtienen todas las filas en ambas tablas, aunque no tengan correspondencia en la otra tabla. Es decir, todos los registros de A y de B aunque no haya correspondencia entre ellos, rellenando con nulos los campos que falten:



Es equivalente a obtener los registros comunes (con un INNER) y luego añadirle los de la tabla A que no tienen correspondencia en la tabla B, con los campos de la tabla vacíos, y los registros de la tabla B que no tienen correspondencia en la tabla A, con los campos de la tabla A vacíos.

Su sintaxis es:

```
SELECT T1.Col1, T1.Col2, T1.Col3, T2.Col7
FROM Tabla1 T1 FULL [OUTER] JOIN Tabla2 T2 ON T1.Col1 = T2.Col1
```

Por ejemplo, en Northwind esta consulta:

```
SELECT OrderID, C.CustomerID, CompanyName, OrderDate
FROM Customers C FULL JOIN Orders O ON C.CustomerID = O.CustomerID
```

nos devuelve nuevamente 832 registros: los clientes y sus pedidos, los clientes sin pedido (hay 2) y los pedidos sin cliente (que en este caso son 0).